LISTA DE EXERCÍCIOS 2

Ponteiros, Structs e Arquivos

Sumário

1 Ponteiros e Alocação Dinâmica **2**
1.1 Exercícios de Programação 2
1.2 Questões de Múltipla Escolha 3
1.3 Encontre o Erro 4

2 Estruturas, typedef e Uniões **5**
2.1 Exercícios de Programação 5
2.2 Questões de Múltipla Escolha 5
2.3 Encontre o Erro 6

3 Processamento de Arquivos **8**
3.1 Exercícios de Programação 8
3.2 Questões de Múltipla Escolha 8
3.3 Encontre o Erro 9

1 Ponteiros e Alocação Dinâmica

1.1 Exercícios de Programação

1.1 - Ordenando três variáveis por referência

Escreva a função `void troca(int *a, int *b)`, que troca os valores apontados por `a` e `b`. Em seguida, usando apenas chamadas a `troca`, escreva `void ordena3(int *a, int *b, int *c)`, que deixa os três valores em ordem crescente: ao final, `*a` $\leq$ `*b` $\leq$ `*c`. No `main()`, leia três inteiros, chame `ordena3` e imprima as três variáveis. Explique, em comentário, por que a função não funcionaria se os parâmetros fossem `int` em vez de `int *`.

1.2 - Mínimo e máximo em uma única função

Escreva a função `void minmax(const int *v, size_t n, int *menor, int *maior)`, que encontra o menor e o maior elemento de um vetor e devolve os dois pelos parâmetros de saída. Percorra o vetor usando **apenas aritmética de ponteiros**: não use colchetes em nenhum ponto da função. Dica: um ponteiro `fim = v + n` marca a posição logo depois do último elemento e serve como condição de parada.

1.3 - Filtrando um vetor dinâmico

Leia um inteiro N, aloque com `malloc` um vetor de N inteiros e preencha-o com valores lidos do teclado. Escreva a função `int *filtraPares(const int *v, size_t n, size_t *tam)`, que devolve um **novo** vetor alocado dinamicamente, com tamanho exato, contendo apenas os elementos pares de `v`, e informa por `tam` quantos são. Se não houver pares, a função devolve `NULL` e `*tam` recebe 0. Teste o retorno de cada `malloc` e libere toda a memória antes de terminar o programa.

1.4 - Matriz dinâmica e transposta

Leia as dimensões m e n e aloque dinamicamente uma matriz de inteiros m×n como um vetor de m ponteiros, cada um apontando para uma linha de n inteiros. Leia os elementos e escreva a função `int **transposta(int **mat, size_t m, size_t n)`, que aloca e devolve uma nova matriz n×m com a transposta. Escreva também `void liberaMatriz(int **mat, size_t linhas)`, que libera primeiro cada linha e depois o vetor de ponteiros. Em comentário, explique por que a ordem das chamadas a `free` importa.

1.5 - Desativar IP

Dado um endereço IP válido (IPv4), retorne uma versão desativada desse endereço IP. Um endereço IP desativado substitui cada ponto "."com "[.]".

```
Exemplo 1:
Entrada: "1.1.1.1"
Saída: "1[.]1[.]1[.]1"


Exemplo 2:
Entrada: "255.100.50.0"
Saída: "255[.]100[.]50[.]0"


Obs.: implemente a função char * defangIPaddr(char * address){ }
```

1.6 - Verificador de palíndromo com ponteiros

Escreva um programa em C para verificar se uma string é palíndromo ou não, mas implementando a verificação em uma função que use ponteiros. Implemente:

`int ehPalindromo(const char *s);`

Na função, use dois ponteiros (um apontando para o início e outro para o final da string) e avance/reduza-os até se cruzarem. Não use índices (colchetes) para comparar caracteres. No `main()`, leia a string (por exemplo, com `fgets`), remova o caractere de nova linha final (`\n`) se existir, chame `ehPalindromo` e informe se é ou não palíndromo.

1.2 Questões de Múltipla Escolha

Questão 1.1

Qual é a saída do trecho abaixo?

```c
int x = 10;
int *p = &x;
*p += 5;
printf("%d %d\n", x, *p);
```

a) 10 15

b) 15 15

c) 10 10

d) O endereço de x seguido de 15

Questão 1.2

Qual é a saída do trecho abaixo?

```c
int v[] = {10, 20, 30, 40};
int *p = v + 1;
printf("%d %d\n", *p + 1, *(p + 1));
```

a) 21 30

b) 30 30

c) 21 21

d) 11 20

Questão 1.3

Qual é a saída do trecho abaixo?

```c
int v[] = {1, 5, 9};
int *p = v;
int a = *p++;
printf("%d %d\n", a, *p);
```

a) 2 1

b) 5 5

c) 1 5

d) 1 1

1.3 Encontre o Erro

Identifique o(s) problema(s) presentes nos códigos abaixo (podem ser erros de sintaxe, de lógica ou comportamento indefinido) e explique como corrigi-los.

Erro 1.1

```
int *p;
*p = 10;
printf("%d\n", *p);
```

Erro 1.2

```
int *criaValor(void) {
    int x = 42;
    return &x;
}

int main(void) {
    int *p = criaValor();
    printf("%d\n", *p);
    return 0;
}
```

Erro 1.3

```
int n = 5;
int *v = malloc(n);
for (int i = 0; i < n; i++)
    v[i] = i * i;
free(v);
printf("Ultimo: %d\n", v[n - 1]);
```

2 Estruturas, typedef e Uniões

2.1 Exercícios de Programação

2.1 - Aritmética de frações

Defina `typedef struct { int num; int den; } Fracao;`. Escreva as funções `Fracao soma(Fracao a, Fracao b)`, `Fracao multiplica(Fracao a, Fracao b)` e `void simplifica(Fracao *f)`, esta última dividindo numerador e denominador pelo seu máximo divisor comum. As duas primeiras devolvem o resultado já simplificado. No `main()`, leia duas frações e imprima a soma e o produto no formato `a/b`. Trate o caso de denominador zero na leitura.

2.2 - Relógio com structs

Defina `struct hora` com os membros hora, minuto e segundo. Escreva: (a) `void avanca(struct hora *h, int segundos)`, que avança o horário na quantidade de segundos informada, ajustando minutos e horas e voltando a 00:00:00 depois de 23:59:59; (b) `int diferenca(struct hora a, struct hora b)`, que devolve a diferença em segundos entre dois horários do mesmo dia; (c) `void imprime(const struct hora *h)`, no formato `HH:MM:SS`. Justifique, em comentário, por que (a) recebe um ponteiro e (b) recebe as structs por valor.

2.3 - Agenda de contatos

Defina `Contato` com nome (até 29 caracteres), telefone (até 15 caracteres) e idade. Leia N contatos para um vetor e implemente um menu com as opções: (1) buscar um contato pelo nome, usando `strcmp`; (2) listar todos em ordem crescente de idade, ordenando o vetor com o método da bolha; (3) listar os contatos com idade dentro de um intervalo lido do teclado; (0) sair. As funções que apenas leem o vetor devem receber `const Contato v[]`.

2.4 - Acervo de biblioteca

Defina `Data` (dia, mês, ano) e `Livro`, com título, autor, ano de publicação, um indicador de empréstimo e a `Data` de devolução prevista. O acervo deve ser um vetor alocado dinamicamente que começa com capacidade 2 e dobra com `realloc` sempre que fica cheio. Implemente funções para: cadastrar um livro; emprestar um livro, registrando a data de devolução; e listar os livros com devolução atrasada em relação a uma data informada. Escreva `int comparaData(Data a, Data b)`, que devolve um valor negativo, zero ou positivo, e use-a na verificação de atraso.

2.2 Questões de Múltipla Escolha

Questão 2.1

Qual é a saída do programa abaixo?

```c
typedef struct { int x; } Caixa;

void muda(Caixa c)  { c.x = 99; }
void mudaP(Caixa *c) { c->x = 7; }

int main(void) {
    Caixa a = {1}, b = {1};
    muda(a);
    mudaP(&b);
    printf("%d %d\n", a.x, b.x);
    return 0;
}
```

a) 99 7

b) 1 1

c) 99 1

d) 1 7

Questão 2.2

Em uma máquina em que `char` ocupa 1 byte, `int` ocupa 4 e `double` ocupa 8, quanto vale `sizeof(union valor)`?

```
union valor {
    char c;
    int i;
    double d;
};
```

a) 13, a soma dos tamanhos dos membros

b) 1, o tamanho do primeiro membro

c) 8, o tamanho do maior membro

d) Depende de qual membro foi gravado por último

Questão 2.3

Considere as declarações abaixo. Qual das expressões **não** compila?

```
struct ponto { int x; int y; };
struct ponto q = {3, 4};
struct ponto *p = &q;
```

a) `p->x`

b) `(*p).x`

c) `*p.x`

d) `q.x`

2.3 Encontre o Erro

Erro 2.1

```
struct ponto {
    int x;
    int y;
}

int main(void) {
    struct ponto a = {1, 2}, b = {1, 2};
    if (a == b)
        printf("Iguais\n");
    return 0;
}
```

Erro 2.2

```
typedef struct {
    char nome[20];
    int idade;
} Pessoa;

Pessoa p;
p.nome = "Ana";
p.idade = 20;
```

Erro 2.3

```
typedef struct {
    char nome[30];
    double nota;
} Aluno;

int main(void) {
    Aluno *a;
    a->nota = 9.5;
    printf("%.1f\n", a->nota);
    return 0;
}
```

3 Processamento de Arquivos

3.1 Exercícios de Programação

3.1 - Cópia em maiúsculas

Escreva um programa que leia do teclado o nome de um arquivo de texto de origem e o de um arquivo de destino, e copie o conteúdo do primeiro para o segundo convertendo todas as letras para maiúsculas, com `fgetc`, `fputc` e `toupper`. Ao final, imprima quantos caracteres e quantas linhas foram copiados. Teste o retorno de `fopen` para os dois arquivos e explique, em comentário, por que a variável que recebe `fgetc` deve ser `int` e não `char`.

3.2 - Boletim a partir de um arquivo

O arquivo `notas.txt` tem uma linha por aluno, no formato `matricula nota1 nota2 nota3`. Escreva um programa que leia esse arquivo com `fscanf`, até que a leitura falhe, e grave em `relatorio.txt` uma linha por aluno com a matrícula, a média com uma casa decimal e a situação: aprovado (média ≥ 7), recuperação ($5 \leq$ média < 7) ou reprovado. Ao final do relatório, grave a média geral da turma e o número de alunos em cada situação. O laço de leitura não pode usar `feof`.

3.3 - Contagem de palavras e caracteres em arquivo

Escreva um programa em C para contar o número de palavras e de caracteres em um arquivo de texto. O programa deve solicitar ao usuário o nome do arquivo a ser aberto (por exemplo, `test1.txt`), abrir o arquivo para leitura, exibir o conteúdo do arquivo na tela e, ao final, imprimir a quantidade total de palavras e a quantidade total de caracteres do arquivo. Considere como palavra uma sequência de caracteres separada por espaços, tabs ou quebras de linha.

3.4 - Excluir uma linha específica de um arquivo

Escreva um programa em C para excluir uma linha específica de um arquivo de texto. Considere que o conteúdo do arquivo `test.txt` seja:

```
test line 1
test line 2
test line 3
test line 4
```

O programa deve solicitar o nome do arquivo a ser aberto e o número da linha a ser removida. Em seguida, exiba o conteúdo atualizado (sem a linha removida). Dica: faça a remoção criando um arquivo temporário e depois substituindo o arquivo original.

3.5 - localizar e substituir palavra

Escreva um programa em C para localizar e substituir uma palavra em um arquivo de texto.

3.2 Questões de Múltipla Escolha

Questão 3.1

O arquivo `dados.txt` já existe e tem 50 linhas. O que acontece com ele após a instrução abaixo?

```c
FILE *f = fopen("dados.txt", "w");
```

a) Ele é aberto e as próximas gravações vão para o fim

b) O conteúdo anterior é apagado e o arquivo começa vazio

c) `fopen` devolve `NULL`, porque o arquivo já existe

d) Ele é aberto apenas para leitura

Questão 3.2

A primeira linha do arquivo `f` é `programacao`, seguida de uma quebra de linha. Qual é o conteúdo de `buf` após o trecho abaixo?

```
char buf[10];
fgets(buf, 10, f);
```

a) `"programacao"`

b) `"programac"`

c) `"programa"`

d) `"programacao\n"`

Questão 3.3

O arquivo binário `f` foi gravado com vários registros do tipo `Reg`, um após o outro. Qual registro é lido pelo trecho abaixo?

```
Reg r;
fseek(f, 2 * sizeof(Reg), SEEK_SET);
fread(&r, sizeof(Reg), 1, f);
```

a) O primeiro

b) O segundo

c) O terceiro

d) Depende da posição em que o arquivo estava antes

3.3 Encontre o Erro

Erro 3.1

```
FILE *f = fopen("texto.txt", "r");
char c;
while ((c = fgetc(f)) != EOF) {
    putchar(c);
}
```

Erro 3.2

```
FILE *f = fopen("numeros.txt", "r");
int n, soma = 0;
while (!feof(f)) {
    fscanf(f, "%d", &n);
    soma += n;
}
printf("Soma = %d\n", soma);
fclose(f);
```

Erro 3.3

```
int *v = malloc(10 * sizeof(int));
for (int i = 0; i < 10; i++)
    v[i] = i;

FILE *f = fopen("dados.bin", "rb");
fwrite(v, sizeof(v), 10, f);
fclose(f);
free(v);
```